Project 1: Burgers' Equation and PINNs

Joaquin Campos

2 April 2026

1 Introduction

In this project, I was tasked with training a PINN to solve the one-dimensional Burgers' equation in a rectangular domain with homogeneous Dirichlet boundary conditions:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}$$

$$t \in (0,1) \quad x \in (-1,1)$$

$$u(x,0) = -\sin(\pi x)$$

$$u(-1,t) = 0 \quad u(1,t) = 0$$

I chose to examine this differential equation since I have not worked much with the theory of nonlinear partial differential equations. This equation was originally used as a simplified model to study fluid flow, by removing the pressure gradient and isolating the competing effects of nonlinear convection $\left(u\frac{\partial u}{\partial x}\right)$ and viscous diffusion $\left(\nu\frac{\partial^2 u}{\partial x^2}\right)$. The modern applications of this equation range from nonlinear acoustics and gas dynamics to macroscopic traffic flow. [3] [1]

To simulate this partial differential equation, I take a classical finite differences approach, as well as a physics-informed neural network (PINN) approach. The PINN method is especially good at handling complex structures, such as nonlinearity, although it is somewhat computationally costly. I also train several PINNs under an ensemble regime to obtain a statistically sound solution. The drawback with this method is the amount of time it takes to complete. For 100 training sessions of the neural network architecture laid out in Section 3, it took an estimated four hours of training time to complete.

2 Mathematics

I would like to spend some time developing the analytical solution to this problem using the Cole-Hopf transformation and Fourier series.

The Burgers' equation is a nonlinear equation, so we cannot immediately solve for the time evolution of the solution with the normal techniques for solving linear PDEs. However, we can convert this equation into a linear equation by the following substitution:

$$u(x,t) = -2\nu \frac{\partial}{\partial x} \log \varphi(x,t)$$

$$\Rightarrow -2\nu \frac{\partial}{\partial x} \left(\frac{1}{\varphi} \left(\frac{\partial \varphi}{\partial t} - \nu \frac{\partial^2 \varphi}{\partial x^2} \right) \right) = 0$$

Integrating with respect to x and rearranging terms gives:

$$\frac{\partial \varphi}{\partial t} - \nu \frac{\partial^2 \varphi}{\partial x^2} = \varphi \frac{df}{dt}$$

where $f(t)$ is an arbitrary function of time. Introducing the transform $\varphi \to \varphi e^f$ transforms this into the standard heat equation:

$$\frac{\partial \varphi}{\partial t} = \nu \frac{\partial^2 \varphi}{\partial x^2}$$

Transforming the initial conditions and boundary conditions gives:

$$\varphi(x,0) = \exp\left(-\frac{2\nu}{\pi} \cos(\pi x) \right) \quad \frac{\partial \varphi}{\partial x}(-1,t) = \frac{\partial \varphi}{\partial x}(1,t) = 0$$

This can then be solved using Fourier series, Sturm-Liouville theory, and modified Bessel functions [5] to obtain the final expansion:

$$\varphi(x,t) = I_0\left(\frac{2\nu}{\pi}\right) + 2\sum_{k=1}^{\infty} (-1)^k I_k\left(\frac{2\nu}{\pi}\right) \cos(k\pi x) e^{-\nu k^2 \pi^2 t}$$

Solving for u then gives:

$$u(x,t) = \frac{4\nu\pi \sum_{k=1\infty} (-1)^k k I_k(\frac{2\nu}{\pi}) \sin(k\pi x) e^{-\nu k^2 \pi^2 t}}{I_0(\frac{2\nu}{\pi}) + 2\sum_{k=1}^{\infty} (-1)^k I_k\left(\frac{2\nu}{\pi}\right) \cos(k\pi x) e^{-\nu k^2 \pi^2 t}}$$

In a separate script, I have plotted the analytical solution of this equation, which can be seen in Figure 1. I did not think to solve this partial differential equation analytically until near the end of this project, so my mean-squared errors are not based on this solution. If I were to start this project over, I would definitely include this in the final code to accurately compare the two methods used.

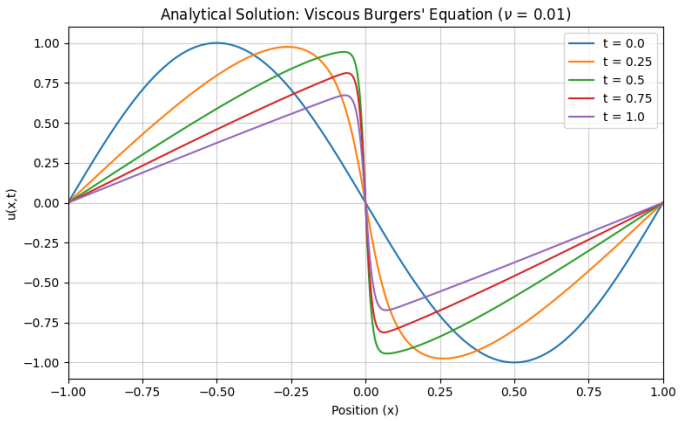


Figure 1: Analytical solution of the 1D viscous Burgers' equation at various time steps ($t = 0.0, 0.25, 0.5, 0.75, 1.0$), demonstrating the formation and viscous dissipation of a shock wave from the initial condition $u(x, 0) = -\sin(\pi x)$.

3 Method

To simulate this partial differential equation, two approaches are taken - finite differences and a PINN method.

The finite differences method is implemented with 200 evenly spaced sampling points in the spatial domain and 1000 points in the temporal domain. I implemented an upwind scheme to account for the travelling waves that occur in the time evolution. [7] This comes from the linear advection equation

$$\frac{\partial u}{\partial t} + a\frac{\partial u}{\partial x} = 0$$

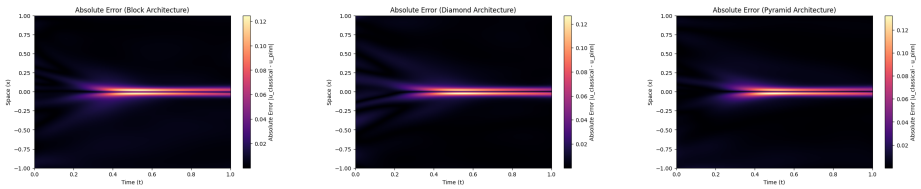
The upwind scheme is given as:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + a\frac{u_i^n - u_{i-1}^n}{\Delta x} = 0 \quad a > 0$$

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + a\frac{u_{i+1}^n - u_i^n}{\Delta x} = 0 \quad a < 0$$

By restricting the computation to only look upwind, the scheme perfectly matches the physical flow of information.

The PINN method is implemented with a block architecture of four hidden layers and 35 nodes per hidden layer, and hyperbolic tangent smoothing functions. I explored several architecture styles, including diamond and pyramid architectures. I ultimately decided to use the block architecture to avoid rank compression and information loss.



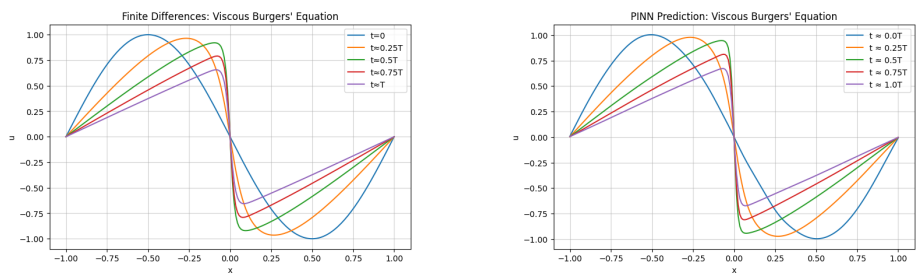
((a)) Block architecture absolute error.

((b)) Diamond architecture absolute error.

((c)) Pyramid architecture absolute error.

Figure 2: Absolute error plots of block, diamond, and pyramid architectures, using the magma colormap [2].

4 Results



((a)) Finite differences plot.

((b)) PINN plot.

Figure 3: Numerical solution and PINN prediction of the 1D viscous Burgers' equation at various time steps ($t = 0.0, 0.25, 0.5, 0.75, 1.0$).

The classical finite differences method ran in 0.48s, and produced the graph seen in Figure 3(a). The PINN method ran in 223.16s, and produced the graph seen in Figure 3(b). Both heatmaps generated can be seen in Figure 4, and the absolute error plot can be seen in Figure 2(a). The absolute error seems to be most concentrated around the shockwave once it has formed, giving absolute errors up to 0.12. The PINN and finite differences have very similar results before the shockwave has formed and outside the local area of the shockwave. This is most likely due to the complex geometry that takes hold, resulting in less accuracy from the classical differences method. Looking at the plots from Figure 3 and Figure 1, it appears that the shockwave is increased in spatial width in the finite differences in comparison to the analytical and PINN methods, but I cannot say this for certain without implementing the analytical method into my original code.

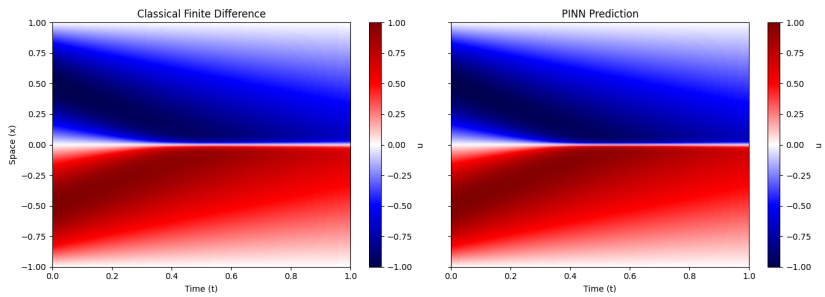


Figure 4: Side-by-side heat maps of the numerical solution and PINN prediction of the 1D viscous Burgers' equaion, using the seismic colormap [2].

Figure 5: Convergence plot for the PINN implementation over 10000 epochs.

The convergence plot displaying the loss at each epoch on a logarithmic scale can be seen in Figure 5. The lowest loss achieved by this PINN was $\sim 3 \times 10^{-4}$, signaling a very accurate neural network prediction. Taking a look at the individual losses, it appears that around 85% of the total loss comes from PDE loss, with the remaining loss being attributed to boundary conditions and initial conditions.

5 Analysis

Both the classical finite differences method and the PINN method have their various strengths and weaknesses. The finite differences method is relatively simple to implement and takes very little time to run. However, this method does not handle complex geometry very well, such as the shockwave present in the Burgers' equation. Contrarily, the PINN method is more complex in implementation and takes considerably more time to run. However, the results can potentially be much more accurate than the finite differences method, especially with more complex structures. Also, the PINN method is very simple to upscale for its intricacy, as the only portion of this code that requires modification are the parameters and the neural network architecture.

For a relatively simple equation such as Laplace's equation, I would likely use a classical approach like finite differences. This is because the geometry is relatively simple and the PINN method takes much longer to run than desirable. For more complicated equations, like the Burgers' equation, I believe the added success from the PINN method is worth the tradeoff of longer runtime.

6 Ensemble Method & Statistical Analysis

To take full advantage of neural networks, I implemented an ensemble method to run 100 trainings in series and optimize to find the best neural network. This idea comes from the statistical nature of neural network training, and the fact that the final loss follows some distribution, discussed briefly in Figure 8. The main drawback for this method is the long time it takes to run. As mentioned in the introduction, running 100 trainings of 10000 epochs took me roughly four hours to complete. Looking at the distribution parameters for each distribution, we see that the location of the MSE distribution is around 0, and the location of the time distribution is around 135. This indicates that the minimum loss possible is 0 and the minimum execution time possible is 135.

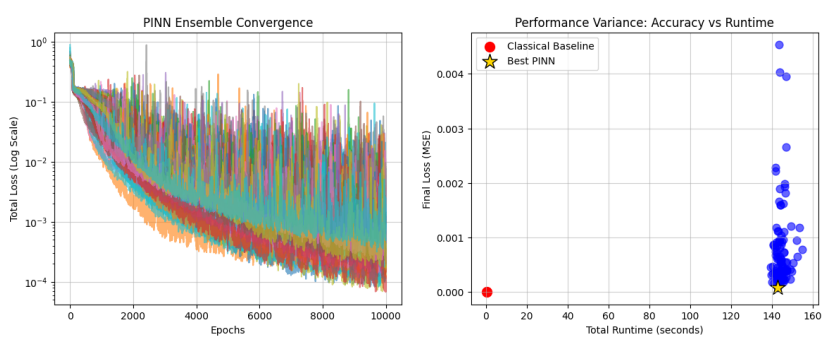


Figure 6: Convergence and performance plots for the ensemble implementation.

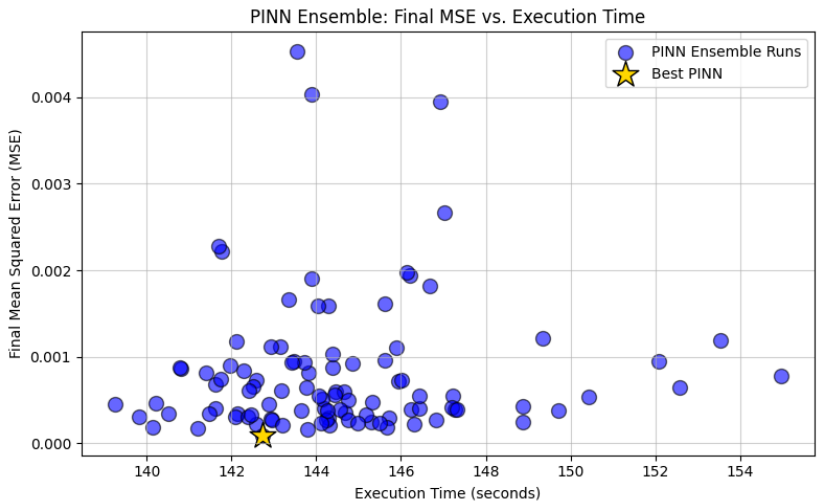
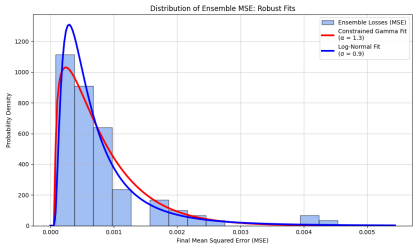
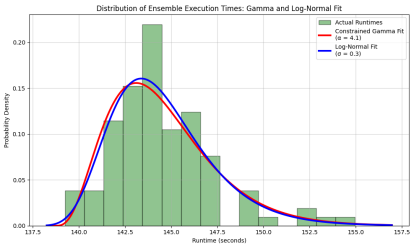


Figure 7: Ensemble performance scatterplot of final loss against time, with correlation coefficient of $r = 0.0628$. This suggests there is no linear correlation between accuracy and runtime.



((a)) Distribution of final loss.



((b)) Distribution of execution time.

Figure 8: Histograms showing the distribution of the loss as well as the execution time for the ensemble method. A log-normal curve and gamma curve are fitted to each distribution. The colors used are cornflower blue and forest green [4].

7 Conclusion

Overall, this project was a success. I was able to implement both methods successfully, and was able to compare and contrast these different approaches to solve this partial differential equation. I was able to then generate visualizations that help intuitively understand the nature of this partial differential equation. This project helps to reinforce the computational methods that are used in modern physics and computational mathematics. In the future, I would likely implement a comparison against the analytical method, and potentially increase the domain to three spatial dimensions. This then allows a natural progression to the Navier-Stokes equations[6], with some given time-varying force field:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \boldsymbol{\nabla})\mathbf{u} = \nu \boldsymbol{\nabla}^2 \mathbf{u} - \frac{1}{\rho}\boldsymbol{\nabla} p + \frac{1}{\rho}\mathbf{F}$$

References

[1] *Burgers' Equation*. URL: https://en.wikipedia.org/wiki/Burgers%27_equation.

[2] *Choosing Colormaps in Matplotlib*. URL: https://matplotlib.org/stable/users/explain/colors/colormaps.html.

[3] Nand Kishor Kumar. *A Review on Burgers' Equations and It's Applications*. URL: https://www.researchgate.net/publication/377931244_A_REVIEW_ON_BURGERS'_EQUATIONS_AND_IT'S_APPLICATIONS.

[4] *List of named colors*. URL: https://matplotlib.org/stable/gallery/color/named_colors.html.

[5] *Modified Bessel Functions of the First Kind*. URL: https://mathworld.wolfram.com/ModifiedBesselFunctionoftheFirstKind.html.

[6] *Navier-Stokes equations: Incompressible flow*. URL: https://en.wikipedia.org/wiki/Navier%E2%80%93Stokes_equations#Incompressible_flow.

[7] *Upwind scheme*. URL: https://en.wikipedia.org/wiki/Upwind_scheme.